

BlindedFlutter

Software Development Plan

Sist Oppdatert: 27.08.2026

Innhold

1. Introduksjon	2
2. Systemoversikt	3
3. Prosjektorganisering.....	4
4. Risikoanalyse.....	5
5. Krav til maskinvare- og programvareressurser	6
6. Arbeidsfordeling.....	7
7. Prosjektplan	8
8. Overvåkings- og rapporteringsmekanismer	9
9. Verktøy	10
Referanser.....	11

1. Introduksjon

BlindedFlutter er en sanntids flerspiller lærings-basert spill utviklet av Aleksander Augestad, Boas Brynsholmen, Christoffer Redmund Tilrem og Housayn Raffea hvor man får svaralternativene før spørsmålet og poeng må bys på det svaret som virker mest logisk.

Systemet til BlindedFlutter målsetter seg på å gi en unik læringsprosess hvor brukere gjetter seg til spørsmålet basert på svaralternativene som er gitt for å maksimere vinnerejanser. Når svaret vises, underbygger det kunnskap som kanskje allerede eksisterer, eller utvikler ny kunnskap. Grunnlaget til BlindedFlutter bygger på spenningen ved gjetting for å inspirere brukere til å lære fra spørsmålet.

En av aspektene til BlindedFlutter er bruk av poeng, ikke bare til å by på spørsmål, men også til å kjøpe hint eller andre fordeler. Dette gjør BlindedFlutter til et betydelig mer involverende og strategisk spill.

Dette dokumentet beskriver planen for design, dokumentering og levering av BlindedFlutter. Prosjektets kildekode ligger i GitHub ved følgende lenke:

<https://github.com/facetype/ReverseKahoot>

2. Systemoversikt

Host-klient: Vises på delt skjerm/prosjektor

Spill-klient: Kjører på hver deltagers enhet

Server: Holder på autorativ spill-tilstand

3. Prosjektorganisering

Agile-modellen ble vurdert som en mulig prosessmodell for prosjektet, på grunn av at det er industristandarden. Agile-modellen innebærer at gruppemedlemmene jobber i iterative skritt ved å dele opp prosjektet i små biter. Modellen fokuserer på at gruppemedlemmene planlegger, jobber sammen, lager små endringer og holder oversikt over prosjektet underveis i prosjektet. For dette prosjektet, så er det satt opp 4 iterasjoner; Sprint1 (Alpha), Sprint2 (Beta), Sprint3 (RC) og Sprint4 (RTM).

For hvert nytt branch som lages i git, så utvikles det nye funksjoner/krav i forhold til 'Software Requirements and Design'-dokumentet. Det er satt opp slik at endringene i et branch må legges til som en pull request i git, hvor det etter hvert må godkjennes av en av utviklerene og merges til main. Dette sikrer for at main branch ikke blir utsatt for uryddig, uoversiktlige og udokumenterte endringer i kildekoden, som blir vanskelig å vedlikeholde senere under utviklingen av prosjektet.

4. Risikoanalyse

Det er et par risikoer ved et system som dette, og flere vil dukke opp. Tabell 4.1 viser til de risikoene som er antatt før prosjektet er satt i gang

Tabell 4.1 Risikoanalyse, før prosjektstart

ID	Risiko	Beskrivelse av risikoen	Sanns.	Kons.	Kategori
1	SQL-Injection	SQL-Injection for å direkte nå databasen	Liten	Svært alvorlig	Kritisk
2	Manglende database plass	For mye lagret på database	Meget liten	Alvorlig	Moderat
3	Upassende Innhold	Mangel på filter for upassende innhold	Liten	Lav	Lav
4	Konkurranse	Konkurranse fra andre liknende quiz apper	Stor	Lav	Moderat
5	Ukryptert brukerinformasjon		Stor	Alvorlig	Kritisk

Disse risikoene er av forskjellige alvorlighetsgrader basert på InnoMed sin mal for risikoanalyse og vil oppdateres jevnt under prosjektets gang.

5. Arbeidsfordeling

- Rollefordeling

Frontend og sanntids UX: Housayn

Backend og spilloikk: Christoffer

Data og QA: Boas

Roller som er fordelt er ikke enkeltpersoners eneste arbeidsoppdrag, men hovedansvar ligger hos den rollen er fordelt under. Oppgaver samarbeides om dersom det er behov for.

- SCRUM-master

Alle gruppelemmer fordeler ansvaret for å bli SCRUM-master (altså; leder av prosjektet) gjennom hver sprint. For det første sprintet, så er Housayn SCRUM-master. For det andre sprintet, så har Boas ansvaret. For det tredje sprintet, så har Aleksander ansvaret. For det fjerde og siste sprint, så har Christoffer ansvaret.

- Code review-praksis

All kode skal gå igjennom pull requests og skal leses og verifiseres av minst en annen person før den merges til main.

6. Prosjektplan

Prosjektplanen i Tabell 6.1 viser til planlagt arbeidsflyt de første 5 ukene, en mer detaljert er vedlagt under filer. Første ukene brukes hovedsakelig til dokumentutforming og planlegging av prosjekt, hovedsakelig SDP+SRP og Stack bestemmelser + Databasebygging.

Tabell 6.6.1 Prosjektplan uke 34-39

	Uke 34	Uke 35	Uke 36	Uke 37	Uke 38	Uke 39
Project Kick-off and planning (SDP)						
Requirements Engineering (SRD)		SDP				
Database Design and Implementation			SRD			
Agile and Scrum				ER diagram		
Software Platforms and Web					Azure DevOps + Alpha	
Project Work						Website

7. Overvåkings- og rapporteringsmekanismer

Kode og versjonskontroll

Prosjektet vil følge prosjektplanen slik den er satt opp i sitt eget dokument.

Ingen kode skal direkte pushes til main, men heller opprettes i en ny branch. Alt som skal merges til main kjøres gjennom pull requests og må være godkjent av minst en annen prosjektarbeider før den kan merges. Alle commits og pull requests skal beskrive hva de gjør.

Rapportskrivning og kildeføring

Tabell 7.1 Referanse- og Kildehenvisningsrettlinjer

Beskrivelse	Stil og/eller metode
Posisjon til tabeller, figurer og bilder og deres tekstbeskrivelser	Midtstilt
Lagring av kilder og kildeføring	Kilder skal lagres i word og føres med word. Mens individuelle gruppemedlemmer undersøker kilder kan de lagre kilder som de ønsker, men dette skal føres inn i word fortløpende dersom kildene brukes i teksten.

8. Verktøy

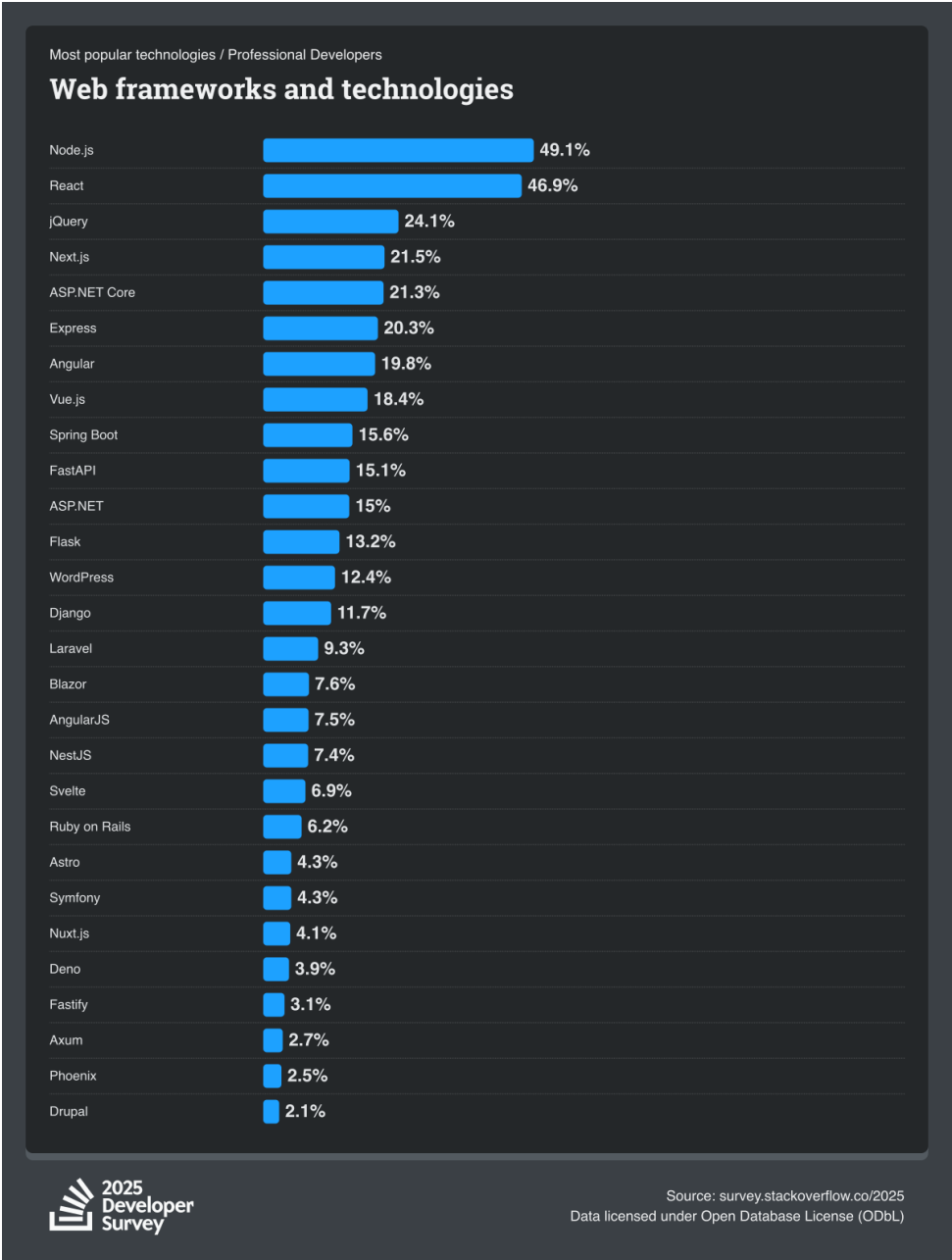
Rammeverk

- Frontend: React, Vite
- Backend: Node, Express, Drizzle
- Database: PostgreSQL

GitHub

KI

Det er veldig mange verktøy som eksisterer for software utvikling i dag. Oftest så er det satt fokus på verktøy som er både enkelt å bruke og lære, gjør jobben til utviklere lettere, og som er mest kjent i dag. Ifølge en spørreundersøkelse utført av stackoverflow i året 2025, så er React brukt av 46.9% utviklere. React er en frontend rammeverk som tilbyr et bibliotek for web- og native brukergrensesnitt. React gjør det mulig å bygge brukergrensesnitt ut ifra komponenter. Komponentene er definert som en del av brukergrensesnittet, hvor den har sin egen logikk og utseende. Et eksempel på en komponent kan være en enkel knapp eller hele siden [1] [2].



Figur 9.1 Bilde fra stackoverflow sin spørreundersøkelse i 2025.

Referanser

[1] Meta, «React,» [Internett]. Available: <https://react.dev/>. [Funnet 27 August 2026].

[2] stackoverflow, «Technology | 2025 Stack Overflow Developer Survey,» 2025. [Internett]. Available: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframe-webframe-prof>. [Funnet 27 August 2026].